

PROGRAMAÇÃO MODULAR

MEMBROS DE CLASSE (*STATIC*)

PROF. JOÃO CARAM

ATRIBUTOS DE CLASSE

2

- Existem atributos que dizem respeito à toda coleção de objetos, e não a um objeto específico.

Ex: Classe *Aluno* → última matrícula gerada
 Classe *Data* → dias de cada ano do mês
 Classe *Inimigo* → número de inimigos vivos no jogo

ATRIBUTOS DE CLASSE

3

- Atributos de classe: compartilhados por todos os objetos daquela classe.
- Escopo local (delimitado pela visibilidade declarada).
- Tempo de vida global.
- Inicializados pelo primeiro objeto ou pelo carregamento da classe.

ATRIBUTOS DE CLASSE (STATIC)

4

- Úteis para:
 - Implementação de contadores e identificadores de autoincremento.
 - definição de constantes de validação.
- Economia de memória:
 - variável é compartilhada pelos objetos de uma classe.
- Declarados com a palavra chave static.

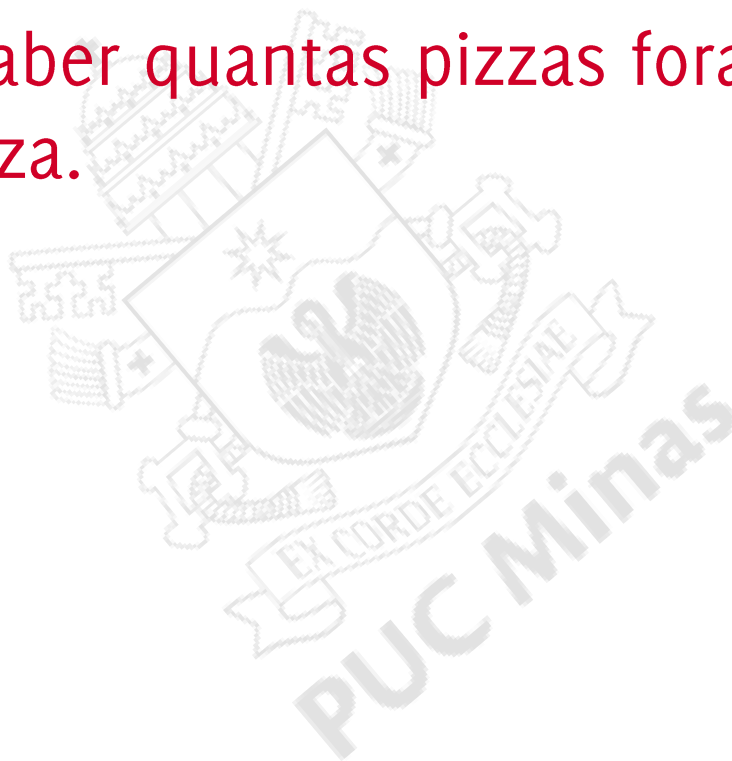


XULAMBS PIZZA!



5

- Queremos saber quantas pizzas foram vendidas na Xulambs Pizza.





XULAMBS PIZZA!



6

- Queremos saber quantas pizzas foram vendidas na Xulambs Pizza.

Pizza

- pizzasVendidas: int
- maxIngredientes: int
- precoBase: double
- valorPorAdicional: double
- quantidadeIngredientes: int

- + Pizza()
- + Pizza(adicionais: int)
- setUp(adicionais: int) : void
- valorAdicionais() : double
- podeAdicionar(quantos: int) : boolean
- + valorFinal() : double
- + adicionarIngredientes(quantos: int) : int
- + notaDeCompra() : String



XULAMBS PIZZA!

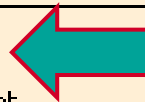


7

- Queremos saber quantas pizzas foram vendidas na Xulambs Pizza.

Pizza

- pizzasVendidas: int
- maxIngredientes: int
- precoBase: double
- valorPorAdicional: double
- quantidadeIngredientes: int



- + Pizza()
- + Pizza(adicionais: int)
- setUp(adicionais: int) : void
- valorAdicionais() : double
- podeAdicionar(quantos: int) : boolean
- + valorFinal() : double
- + adicionarIngredientes(quantos: int) : int
- + notaDeCompra() : String

A large, faint watermark of the PUC Minas logo is centered in the background. It features a shield with a star, a banner with the motto "EX CORDE ECCLESIAE", and the text "PUC Minas" below it.

{ Quero ver código!! }

PIZZA

```
private void setUp(int adicionais){  
    maxIngredientes = 8;  
    precoBase = 29d;  
    valorPorAdicional = 5d;  
    quantidadeIngredientes = 0;  
    adicionarIngredientes(adicionais);  
    pizzasVendidas++;  
}
```

PIZZA

```
private void setUp(int adicionais){  
    maxIngredientes = 8;  
    precoBase = 29d;  
    valorPorAdicional = 5d;  
    quantidadeIngredientes = 0;  
    adicionarIngredientes(adicionais);  
    pizzasVendidas++;  
}
```



XULAMBS PIZZA!



11

■ Pizza:

- Sem sabores pré-definidos.
- Preço inicial de R\$29.
- Pode ter até 8 ingredientes adicionais.
- Adicionais têm custo fixo: cada um custa R\$5.
- Cupom de venda (relatório), contendo sua descrição e valores a serem pagos.

Valem para
todas as pizzas



XULAMBS PIZZA!



12

▣ Pizza:

- ▣ Sem sabores pré-definidos.
- ▣ Preço inicial de R\$29.
- ▣ Pode ter até 8 ingredientes adicionais.
- ▣ Adicionais têm custo fixo: cada um custa R\$5.
- ▣ Cupom de venda (relatório), contendo sua descrição e valores a serem pagos.

Constantes

CLASSE PIZZA

13

Pizza

- maxIngredientes: int
- precoBase: double
- valorPorAdicional: double
- quantidadeIngredientes: int

- + Pizza()
- valorAdicionais() : double
- podeAlterar(quantos: int) : boolean
- + valorFinal() : double
- + adicionarIngredientes(quantos: int) : int
- + retirarIngredientes(quantos: int) : int
- + notaDeCompra() : String



Pizza

- MAX INGREDIENTES: int
- PRECO BASE: double
- VALOR POR ADICIONAL: double
- quantidadeIngredientes: int

- + Pizza()
- valorAdicionais() : double
- podeAlterar(quantos: int) : boolean
- + valorFinal() : double
- + adicionarIngredientes(quantos: int) : int
- + retirarIngredientes(quantos: int) : int
- + notaDeCompra() : String

A large, faint watermark of the PUC Minas logo is centered in the background. It features a shield with a star, a banner with the motto 'EX CORDE ECCLESIAE', and the text 'PUC Minas' below it.

{ Quero ver código!! }

CLASSE PIZZA

```
class Pizza{  
    private static final int MAX_INGREDIENTES = 8;  
    private static final double PRECO_BASE = 29d;  
    private static final double VALOR_ADICIONAL = 5d;  
    private int quantidadeIngredientes  
    ...  
}
```

CLASSE PIZZA

```
class Pizza{  
    private static final int MAX_INGREDIENTES = 8;  
    private static final double PRECO_BASE = 29d;  
    private static final double VALOR_ADICIONAL = 5d;  
    private int quantidadeIngredientes  
    ...  
}
```


CLASSE PIZZA

```
class Pizza{  
    private static final int MAX_INGREDIENTES;  
    private static final double PRECO_BASE;  
    private static final double VALOR_ADICIONAL;  
    private int quantidadeIngredientes  
    static{  
        MAX_INGREDIENTES = 8;  
        PRECO_BASE = 29d;  
        VALOR_ADICIONAL = 5d;  
    }  
}
```

CLASSE PIZZA

```
class Pizza{  
    private double valorAdicionais(){  
        return quantidadeIngredientes * VALOR_ADICIONAL;  
    }  
    public double valorFinal(){  
        return PRECO_BASE + valorAdicionais();  
    }  
}
```

CLASSE PIZZA

```
class Pizza{  
    private double valorAdicionais(){  
        return quantidadeIngredientes * VALOR_ADICIONAL;  
    }  
    public double valorFinal(){  
        return PRECO_BASE + valorAdicionais();  
    }  
}
```

20

BÔNUS: EXERCÍCIO/EXEMPLO ALUNO



EXEMPLO/EXERCÍCIO: ALUNO

21

- Um aluno de um curso livre é representado por seu nome e número de matrícula.
- Existem vários cursos, mas o aluno só se matricula em um de cada vez.
- O aluno faz 4 avaliações por semestre.
- O aluno será aprovado se a nota final for maior ou igual a 60 e tiver frequência de no mínimo 75%.

EXEMPLO/EXERCÍCIO: ALUNO

22

- Nome e número de matrícula.
- Um curso por vez.
- O aluno faz 4 avaliações por semestre.
- Aprovado: nota final é ao menos 60 e frequência ao menos 75%.

EXEMPLO/EXERCÍCIO: ALUNO

23

- Nome e número de matrícula.
- Um curso por vez.
- O aluno faz 4 avaliações por semestre.
- Aprovado: nota final é ao menos 60 e frequência ao menos 75%.

Quem define o número de matrícula?

EXEMPLO/EXERCÍCIO: ALUNO

24

- ▣ Nome e número de matrícula.
- ▣ Um curso por vez.
- ▣ O aluno faz 4 avaliações por semestre.
- ▣ Aprovado: nota final é ao menos 60 e frequência ao menos 75%.

Valores são iguais para todos os alunos?

EXEMPLO/EXERCÍCIO: ALUNO

25

- Nome e número de matrícula.
- Um curso por vez.
- O aluno faz 4 avaliações por semestre.
- Aprovado: nota final é ao menos 60 e frequência ao menos 75%.

Como calcular a nota final?

Como calcular a frequência?

MÉTODOS DE CLASSE (STATIC)

26

- Também podemos ter métodos que não precisam ou não devem acessar dados particulares de objetos

- Ex: Classe *Integer*: Conversão string -> inteiro

Classe *Aluno*: comparar dois alunos

Classe *App*: método main

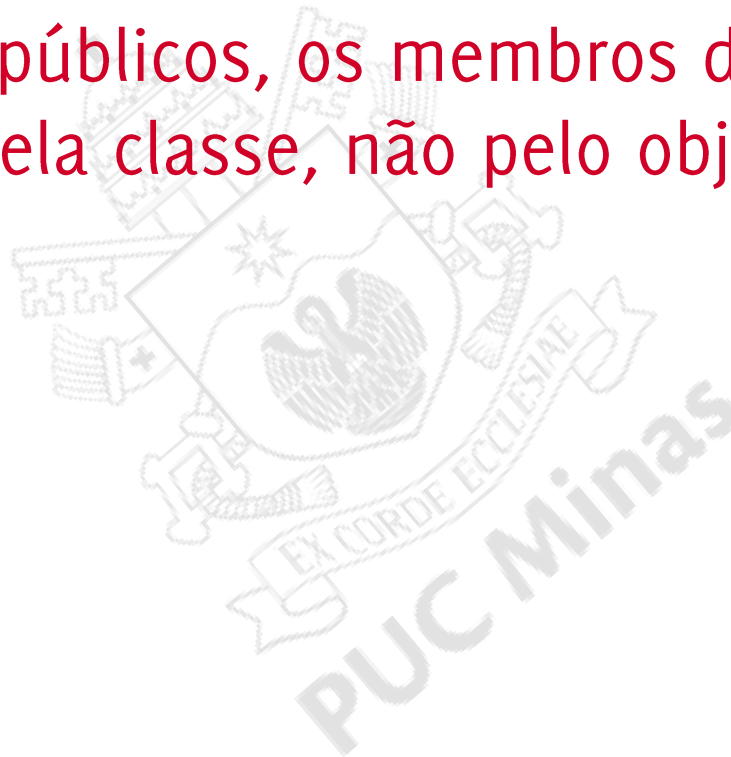
CLASSE ALUNO

```
class Aluno{  
    ...  
    public static Aluno maiorNota(Aluno a1, Aluno a2){  
        if(a1.notaFinal() > a2.notaFinal())  
            return a1;  
        else  
            return a2;  
    }  
    ...  
}
```

MEMBROS STATIC

28

- Caso sejam públicos, os membros de classe são acessados pela classe, não pelo objeto.



CLASSE ALUNO

```
class Aluno{  
    private static int proxMatricula;  
    public static final double NOTAAPROVACAO;  
    public static final double FREQMINIMA;  
    private int final matricula;  
    ...  
}
```

CLASSE ALUNO

```
class App{  
    public static void main (String args[] ) {  
        Aluno aluno1 = new Aluno("42", "Ada Lovelace");  
        System.out.print("Nota para aprovação: ");  
        System.out.println(Aluno.NOTAAPROVACAO);  
        System.out.print("Nota obtida: ");  
        System.out.println(aluno1.notaFinal());  
    }  
}
```

CLASSE ALUNO

```
class App{  
    public static void main (String args[] ) {  
        Aluno aluno1 = new Aluno("42", "Ada Lovelace");  
        System.out.print("Nota para aprovação: ");  
        System.out.println(Aluno.NOTAAPROVACAO);  
        System.out.print("Nota obtida: ");  
        System.out.println(aluno1.notaFinal());  
    }  
}
```

CLASSE ALUNO

```
class App{  
    public static void main (String args[] ) {  
        Aluno aluno1 = new Aluno("42", "Ada Lovelace");  
        Aluno aluno2 = new Aluno("77", "Dorothy Vaughn");  
        aluno1.lancarNota(22.5);  
        aluno2.lancarNota(24.9);  
        Aluno maior = Aluno.maiorNota(aluno1, aluno2);  
        System.out.print("Maior nota obtida: ");  
        System.out.println(maior.notaFinal());  
    }  
}
```


CLASSES ESTÁTICAS

33

- Classes que contêm apenas atributos e métodos de classe podem ser chamadas classes estáticas.
- Ex:
 - *bibliotecas* de funções → System, Math...
 - manipulação de tipos → Integer.parseInt

OBRIGADO.

DÚVIDAS?